

CS 3960 (3)

Vibe Coding

Spring 2026, *content* Pavel Panchekha and John Regehr, *illustrations* Gemini 3 Nano Banana Pro

- How's the latency assignment coming along?
- Reading assignment due Friday
 - Sections 1 and 2 of this paper: [https://
www.cs.tufts.edu/~nr/cs257/archive/john-hughes/
quick.pdf](https://www.cs.tufts.edu/~nr/cs257/archive/john-hughes/quick.pdf)
 - Answer the questions
 - The paper uses the word “quantify”



Today:

- Finish up fuzzing
- Property-based testing
- Assertions
- Defensive programming

These things are all distinct, but related!

Fuzzing Problem #3: The Oracle Problem

How can we tell whether some particular test case triggered a bug in our system?

We've already looked at oracles—
does random testing make the oracle
problem easier or harder?





Function-inverse pairs:

Your code has two operations that
undo each other

So, you test that the undo each other!

**Examples of function-inverse
pairs?**



Encrypt / decrypt

Compress / decompress

Parse / print

Assemble / disassemble

**Sometimes you have to add an
extra round trip!**

Metamorphic testing:

Mutate a test case, but don't
change its meaning

Examples?





Oracles for random testing:
**There's a lot of room for
creativity. Be creative.**

Practical advice for fuzzing:

Start early. If you have a simple system, write a simple fuzzer for it. Then evolve both the system and the fuzzer.

Sometimes people wait for a system to be released, before fuzzing it. This is a bad idea!

In a standard test suite, we have give the system some given input, and we expect a specific output.

In **property-based testing**, we define properties that the system is supposed to have, and see if they ever get violated (typically, using a fuzzer).





Where do properties come from?

They come from the goals of the code you are writing.

If something has to be true at some point in the code, that's a property!

Let's be concrete!

For Python, the most well-known
property-based testing library is
Hypothesis

Let's write properties for a Shell sort
(unrelated to shellfish — this was
named after its inventor Donald Shell)




```
def shell_sort(values: Sequence[T]) -> list[T]:
    arr = list(values)
    n = len(arr)
    gap = n // 2
    while gap > 0:
        for i in range(gap, n):
            temp = arr[i]
            j = i
            while j >= gap and arr[j - gap] > temp:
                arr[j] = arr[j - gap]
                j -= gap
            arr[j] = temp
        gap //= 2
    return arr
```

**What properties do
we expect a sort to
have?**

The output of a sort should be in order:

```
@given(st.lists(st.integers()))  
def output_is_sorted(values: list[int]) -> None:  
    result = shell_sort(values)  
    assert all(result[i] <= result[i + 1] \  
               for i in range(len(result) - 1))
```

The output of a sort should be a permutation of the input:

```
@given(st.lists(st.integers()))
def test_output_is_permutation(values: list[int]) -> None:
    result = shell_sort(values)
    assert len(result) == len(values)
    counts: dict[int, int] = {}
    for value in values:
        counts[value] = counts.get(value, 0) + 1
    for value in result:
        counts[value] = counts.get(value, 0) - 1
    assert all(count == 0 for count in counts.values())
```

Now we run Hypothesis:

```
$ python3 -m pytest -q shell_sort_properties.py
```

```
..
```

```
[100%]
```

```
2 passed in 0.14s
```

```
$
```

That's it! Property-based testing is super easy! Your in-class exercise later today will be to use Hypothesis.



Hypothesis properties live outside of the code — this is convenient for test-case generation

But we often want properties to live inside the code.

These are assertions.

When do you write an assertion?

Any time there's something that must be true, if your code is working properly.



For example, in a Unix program:

```
int result = close(fd);  
assert(result == 0);
```

Is this a good assertion?





Here's another assertion:

```
int y = a + b + c;
```

```
assert(y == (a + b + c));
```

Is this a good assertion?



Here's another assertion:

```
int result =  
open("file.txt", O_RDWR);  
assert(result != -1);
```

Is this a good assertion?

Important:

Assertions are not for error checking.

They're for checking internal consistency of your code.



Assertions come from:

- Basic math
- Preconditions
- Postconditions
- Invariants
- The specification





Important: **Assertions must never have side effects**

In some cases, a “release” build of our software will have assertions stripped out

Is that what we want?

It will make me happy if you learn to
use assertions in your code

This will make you happy, too (but on
a longer time frame)

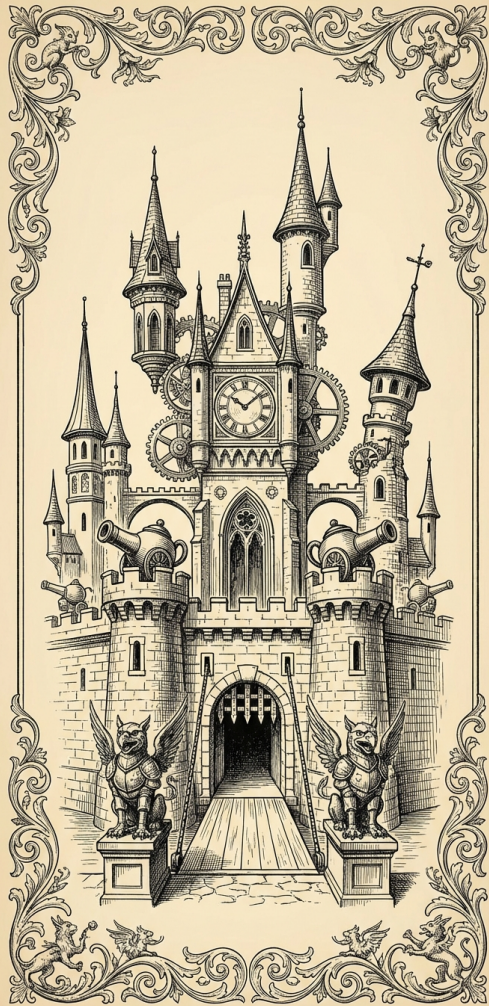
One assertion per 20 lines of code is
not too much



More than halfway through spring semester,
I hope we can all agree that:

- The AI sometimes doesn't understand what you want
- Even when the AI seems to understand what you want, it sometimes does the wrong thing
- We don't understand vibe code as well as code we wrote by hand





Let's talk about **defensive programming**

First, defensive driving:

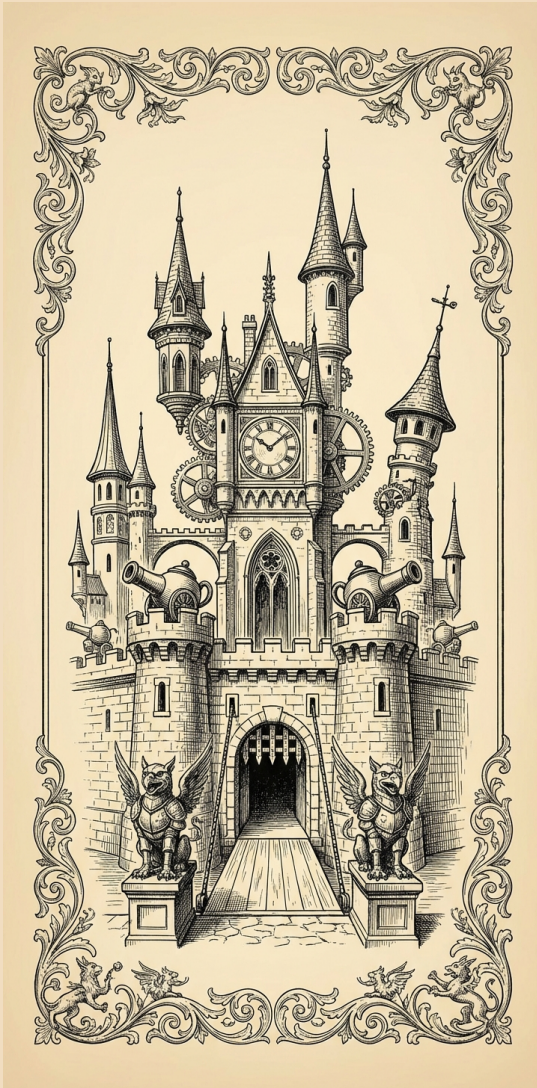
- Anticipate things going wrong and avoid them in advance
- Have your vehicle in a place where things are less likely to go wrong
- Don't drive super fast

Essence of defensive driving:

Assume things will go wrong. Make it unlikely to go wrong for you.

Essence of defensive programming: Assume things will go wrong. Make it unlikely to go wrong for your software.





Defensive programming:

Make as few assumptions as possible;
write code to explicitly check
assumptions

When something goes wrong, fail
rapidly and loudly — use assertions!

Defensive programming:

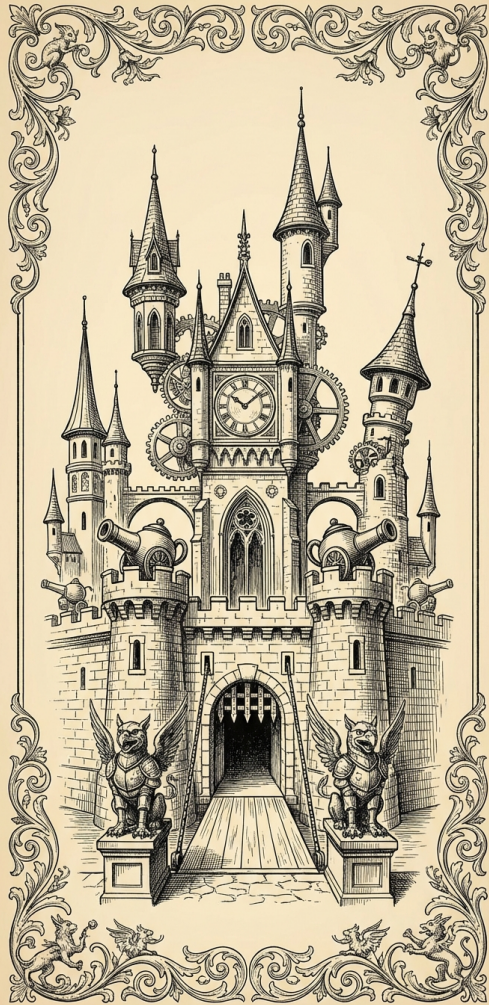
Be paranoid about interfaces

Understand your trusted computing base (TCB)

All of this is distinct from testing!

But these things do connect up





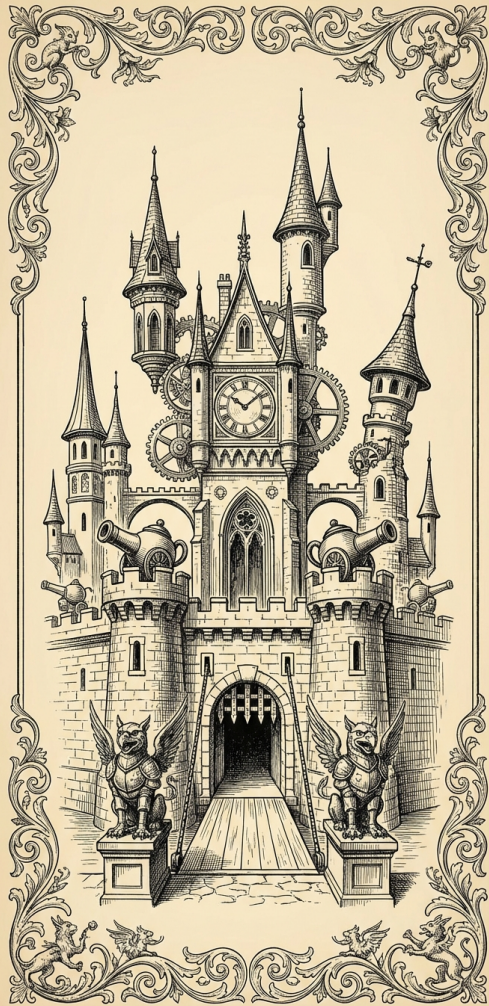
Starting points for defensive programming:

- Use a language with a strong type system
- Turn on optional language-based defenses (compiler warnings, gradual types, sanitizers, etc.)

More defensive programming:

- Write lots of assertions!
- If a function call might fail, you must check for that
- Write lots of tests
- Do property-based testing





There has never been a shortage of people who can type code.

There has always been (and always will be) a shortage of people who can create and maintain working, useful systems.

Being this kind of person is your job insurance.

More reading:

How to Fuzz an ADT Implementation

Write Fuzzable Code

Use of Assertions

The Fuzzing Book

- In the “syllabus” repo in the class GitHub org, find this file: `slides/a6-shellsort-buggy.py`
- This is the Shell sort from earlier
 - Except that we’ve added a bug!
- Add Hypothesis support for the sort properties
- Paste your modified program + output showing failure(s) into the assignment response